

Ensemble: the path to production

Prepared 7 September 2026 | RG-GitUser/ensemble | main at c059e23

Where things stand

PR #3 is merged. main carries the 57 audit findings, RTMPS stream keys, the social connection check, per-creator relay egress metering, and sign-in with Google, Microsoft and Yahoo. CI is green on it.

The hardening branch question is settled. Roughly 25 of its fixes were checked against main one at a time. Every one was already there, and where the two differed main's was better placed: referrer sanitising moved inside recordPageView so no caller can skip it, stream-key injection closed with NUL-delimited targets rather than base64, the unpublished-metadata gate moved into siteForHost so the title and body cannot disagree. A test merge conflicted in 30 of 32 files, which is what two parallel rewrites of the same code look like. It is archived as the tag archive/hardening and the branch is deleted. Four other merged branches were removed with it.

Two PRs are open and green. #4 fixes a test runner that could not execute on Windows at all. #5 closes two places the app promised something no configuration could deliver.

Still true from the last checklist: nothing enforces that CI passes before a merge, and none of the non-code launch blockers have moved.

Part 1 - Merge what is ready

Ten minutes. Everything after this assumes main is current.

- ☐ **Merge PR #4 - the test runner**
npm test printed 'Test sources failed to compile' on a tree that compiles clean, because the compile never ran: npx on Windows is npx.cmd and spawnSync will not resolve .cmd. Until this merges, the suite cannot run on the machine the work happens on.
- ☐ **Merge PR #5 - two promises the app could not keep**
TikTok advertised one-click publishing that no environment value could unlock, and .env.example shipped an RFC 5737 documentation IP as live DNS instructions.
- ☐ **Merge this checklist**
The PDF and the script that renders it. A checked-in binary nobody can regenerate goes stale silently, which is the failure this repo keeps finding in its own comments.
- ☐ **Make the CI check required on main**
Settings -> Branches -> add a rule for main -> require the 'check' status. Both open PRs are mergeable right now with nothing verifying that CI passed. They happen to be green. The next one might not be.

Part 2 - The droplet's .env

New in this revision. The app was run in production mode and every environment gate read in the code. These are the values whose absence does not announce itself.

- ☐ **APP_URL is the public origin**
It defaults to http://localhost:3000, and that default is what builds password-reset and account-recovery links. Unset in production, the mail sends successfully and every recipient gets a link to their own machine. Nothing errors.
`APP_URL=https://ensemble.it.com`

☐ **RESEND_API_KEY and MAIL_FROM are set**

All outbound mail is gated on RESEND_API_KEY. Without it there is no password reset, no account recovery and no newsletters - the Forgot page says so on screen. In development that is a banner; in production it means a locked-out customer has no way back in.

```
RESEND_API_KEY=re... # domain verified at resend.com
MAIL_FROM=Ensemble <news@ensemble.it.com>
AUTH_MAIL_FROM=Ensemble <noreply@ensemble.it.com>
```

☐ **PLATFORM_HOSTS lists every hostname Caddy routes to the app**

Defaults to localhost,127.0.0.1. Sites.* included, or a direct visit to the CNAME target 404s as an unknown customer domain.

```
PLATFORM_HOSTS=ensemble.it.com,www.ensemble.it.com,sites.ensemble.it.com
```

☐ **DOMAIN_A_RECORD is uncommented and set to the reserved IP**

It now ships commented out on purpose. The value that used to ship, 203.0.113.10, is documentation space routed by nobody - so forgetting this edit did not fail, it told every creator to point their apex A record at an address that never answers. Commented, the DNS step hides instead.

```
#DOMAIN_A_RECORD=203.0.113.10 <- as shipped
DOMAIN_A_RECORD=104.248.107.30 <- the droplet's reserved IP
```

☐ **Stripe keys are set, or billing is disabled on purpose**

With NODE_ENV=production and no STRIPE_SECRET_KEY, billing fails closed and logs why. That is the fix for free-Enterprise-by-default. Paid features are then unavailable to everyone until one of the two lines below exists.

```
STRIPE_SECRET_KEY=sk_live...
STRIPE_WEBHOOK_SECRET=whsec...
# or, deliberately:
ENSEMBLE_BILLING_DISABLED=1
```

☐ **ADMIN_PASSWORD is still commented out**

It is seeded once and the mistake is permanent, against an admin address published in this repo. Unset, a random password is generated and printed to the log on first start.

```
grep -n '^ADMIN_PASSWORD' /srv/ensemble/.env # expect: no output
journalctl -u ensemble | grep -i 'admin password'
```

☐ **Optional, and safe to leave unset**

Each is inert when absent rather than broken. SSO providers missing either value are simply not rendered on the login page. The relay needs both of its values or neither.

```
GOOGLE_/MICROSOFT_/YAHOO_CLIENT_ID + _CLIENT_SECRET # sign-in
THREADS_/INSTAGRAM_/FACEBOOK_/PINTEREST_/REDDIT_APP_ID + _APP_SECRET
LIVE_INGEST_URL + LIVE_HOOK_SECRET # relay; both or neither
LIVE_EGRESS_BYTES_PER_SITE # default 250 GB/creator/month
INTUIT_CLIENT_ID + INTUIT_CLIENT_SECRET # QuickBooks
STRIPE_AUTOMATIC_TAX=1 # only after Tax is configured
WIP_MODE=1 # holding page
```

Set Stripe Tax up in the dashboard BEFORE setting the flag. In that order. STRIPE_AUTOMATIC_TAX=1 before Tax is configured makes every checkout fail.

☐ **The file is not world-readable**

Every secret lives in it. The default umask leaves it mode 644.

```
chmod 600 /srv/ensemble/.env
```

```
stat -c '%a %U:%G' /srv/ensemble/.env # expect: 600 ensemble:ensemble
```

Part 3 - Launch blockers that are not code

None of these can be written for you. The first is the longest pole - start it today.

☐ **Legal review by a qualified person**

Terms with governing law, dispute resolution and a refund policy - you bill monthly and deliberately do not prorate, which has to be somewhere a customer agreed to. A privacy page naming Stripe, Resend and DigitalOcean as subprocessors, with retention periods and data-subject rights. An acceptable-use policy with an abuse contact and a DMCA process.

☐ **Stripe: live mode and one full lifecycle**

Register the webhook against the six events billing.ts handles, then run it end to end: subscribe, upgrade, downgrade, fail a payment, cancel, delete the account - and confirm the card actually stops. That last step is the finding most likely to become a chargeback.

☐ **Restore a backup onto a scratch droplet**

This has only ever been done on a laptop. It is the one step that turns a backup from a belief into a backup. Do it once before launch and once a quarter after.

☐ **Make one real post per social provider**

OAuth apps are on production now, but that path has never been proven against a live API. Treat the first post to each provider as a test, not as a launch. The 'Check connection' button on Integrations verifies a token without publishing anything.

☐ **Register the SSO apps, if you want sign-in live**

Optional. Any provider missing either value is simply not rendered on the login page, so leaving these unset keeps email-and-password as the only way in. Redirect URIs are in .env.example.

Part 4 - Server security checklist

Run as root on the droplet unless a line says otherwise. Commands are from DEPLOY.md; each box is one thing to confirm rather than assume.

A. Accounts and firewall

☐ **Service account exists and is NOT in sudo**

Compromising the app must not be the same as owning the box. Administer as root or from your own sudo-capable login.

```
adduser ensemble
```

```
groups ensemble # must NOT list sudo
```

```
deluser ensemble sudo # only if it does
```

☐ **Firewall allows only SSH, HTTP and HTTPS**

```
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw enable
```

```
ufw status verbose
```

B. Secrets and service hardening

☐ The relay has its OWN env file, not the app's

EnvironmentFile loads a whole file, not one line. Pointing the relay at the app's .env put every platform secret into its environment and into every ffmpeg, curl and jq child it spawns.

```
printf 'LIVE_HOOK_SECRET=%s\n' 'the-value-you-generated' > /etc/ensemble-relay.env
chown root:ensemble /etc/ensemble-relay.env
chmod 640 /etc/ensemble-relay.env
```

☐ systemd sandboxing is actually in effect

These are set in deploy/ensemble.service. Confirm the running unit picked them up rather than trusting the file on disk.

```
systemctl show ensemble -p NoNewPrivileges -p ProtectSystem \
-p MemoryMax -p StartLimitBurst -p StartLimitIntervalSec
# expect NoNewPrivileges=yes, ProtectSystem=strict, a real MemoryMax
```

C. Backups

☐ sqlite3 is installed

Not optional: backup.sh and the whole documented restore shell out to it. Without it the nightly job died on its first command every night, silently, under set -euo pipefail.

```
apt-get install -y sqlite3
command -v sqlite3
```

☐ The backups directory exists BEFORE the crontab line

Load-bearing ordering. Cron's shell opens the redirect before the script runs, so on a fresh droplet the job never executed - and because the failure was in the redirect, there was no log to notice it in.

```
chmod +x /srv/ensemble/scripts/backup.sh
mkdir -p /srv/ensemble/backups
crontab -e -u ensemble
0 4 * * * /srv/ensemble/scripts/backup.sh >> /srv/ensemble/backups/backup.log 2>&1
```

☐ rclone is configured as the SERVICE user, not root

The cron job runs as ensemble and rclone reads its config from the running user's home. A remote configured as root is invisible to the job that needs it.

```
sudo -iu ensemble rclone config
grep -n '^BACKUP_REMOTE' /srv/ensemble/.env
```

Never back up the database with cp. It runs in WAL mode, so at any moment the most recent writes are in app.db-wal rather than app.db. Copying app.db alone gets you an almost empty database that still opens cleanly - the worst kind of bad backup. Use sqlite3 .backup, or take app.db, app.db-wal and app.db-shm together.

D. Live relay

☐ RTMPS is open; plain RTMP is closed to the world

The ingest key travels in the URL path in cleartext, and creators stream from venue and hotel wifi. Only the local ffmpeg forwarders use 1935, over loopback, which the firewall does not touch.

```
ufw allow 1936/tcp
ufw deny 1935/tcp
chmod +x /srv/ensemble/deploy/live-push.sh
```

E. Monitoring

☐ **Caddy's log directory exists**

```
mkdir -p /var/log/caddy && chown caddy:caddy /var/log/caddy
```

☐ **An uptime check points at the health route and wakes a phone**

1-5 minute interval. It touches the database, so it distinguishes 'up' from 'up and returning 500 to everyone' - which nothing outside the box could previously tell.

```
https://ensemble.it.com/api/health
```

☐ **You know how to read the logs when it goes wrong**

```
systemctl status ensemble # 'failed', not endlessly 'activating'
```

```
journalctl -u ensemble -n 50
```

```
tail -f /var/log/caddy/access.log
```

Part 5 - Deploy, and be able to undo it

main is 17 commits ahead of what is running. Do not deploy until Part 2 is done: this release is the one where billing fails closed.

☐ **Snapshot first, always**

npm ci deletes node_modules before it starts and next build overwrites .next, on a box that may need swap to build at all. A build that OOMs halfway leaves no working previous version to fall back to.

```
sudo -iu ensemble bash -lc 'cd /srv/ensemble && cp -a .next ../next-prev \
&& git rev-parse HEAD > ../deployed-sha'
```

☐ **Update as the service user, never as root**

The repo is owned by ensemble, and building as root leaves root-owned .next/ and node_modules/ the service cannot write to.

```
sudo -iu ensemble bash -lc 'cd /srv/ensemble && git pull && npm ci && npm run build'
systemctl restart ensemble
curl -fsS https://ensemble.it.com/api/health # expect: {"status":"ok"}
```

☐ **Know the rollback before you need it**

```
sudo -iu ensemble bash -lc 'cd /srv/ensemble \
&& git checkout $(cat ../deployed-sha) && rm -rf .next && cp -a ../next-prev .next'
systemctl restart ensemble
```

☐ **Take the pending reboot at a time you choose**

The droplet login banner reports a required restart. Better now, deliberately, than halfway through a deploy.

F. Verify from OUTSIDE the box

Run these from your laptop, not the droplet. Several of these controls only mean anything when tested from the internet.

☐ **Health route answers**

```
curl -s -o /dev/null -w '%{http_code}\n' https://ensemble.it.com/api/health # 200
```

☐ **Security headers are present and http redirects to https**

```
curl -sI https://ensemble.it.com | grep -iE 'strict-transport|x-content-type'
curl -sI http://ensemble.it.com | head -1 # expect 301/308
```

☐ **The on-demand-TLS ask endpoint is blocked at the edge**

Blocking it is what stops outsiders enumerating your customers.

```
curl -s -o /dev/null -w '%{http_code}\n' \
'https://ensemble.it.com/api/domains/check?domain=example.com' # not 200
```

☐ **The live hook routes refuse an unauthenticated caller**

```
curl -s -o /dev/null -w '%{http_code}\n' \
'https://ensemble.it.com/api/live/targets?key=whatever' # expect 401
```

☒ **A password reset arrives, and its link points at the real domain**

The single check that proves Part 2 landed: mail configured, and APP_URL not left on its localhost default.

Part 6 - Still open in the code

Not blocking a launch, but worth knowing they are outstanding.

☐ **Newsletter sending from the creator's own mailbox**

The other half of the email work. Gmail caps at about 500 recipients/day (2,000 on Workspace), Outlook about 300, Yahoo similar, and all three treat list mail through a personal mailbox as a ToS matter. Proposed design: show the cap against their real subscriber count before they send, refuse rather than half-deliver, keep Resend for anyone above it.

☐ **Relay push retry**

If one ffmpeg dies on a platform hiccup, that destination is gone for the rest of the stream and the creator is not told. Deliberately not shipped untested: it is signal handling in a live path, and getting it wrong leaves stray pushes holding a frozen frame on someone's channel. Needs a real MediaMTX and a real stream to verify.

☐ **Upload filenames are still guessable**

theme---. Renaming to random ids is a migration touching every stored config value, and the exposure is a draft's imagery rather than anything executable. Documented in the route and left deliberately.

☐ **TikTok publishing - resolved in PR #5, but not implemented**

It no longer advertises one-click publishing it cannot do. Wiring it for real needs a provider written against an API that wants video uploads and an audited app, which is not a gap the runbook can close. Flip authType back to 'oauth' in the same commit that adds the provider.

Every command above is from DEPLOY.md as it stands on main. Items marked with a red box are the ones that cost real money or real trust if skipped.